

SOUTENANCE · VEILLE TECHNOLOGIQUE

Vue + NestJS

Tableau Kanban



Vue dans mon appli : ce que ça donne

CE QUE J'UTILISE DE VUE

Composants : colonne, carte, fenêtre d'édition

Réactivité : ref et computed

Store Pinia : données partagées (auth, board)

Routeur : 8 routes, protégées par permission

Composable : filtres des cartes

Build : 66,7 Ko compressés (page d'entrée)

POURQUOI C'EST BIEN

- Template proche du HTML : lisible par toute l'équipe
- ref / computed : l'écran se met à jour tout seul
- Pinia et Vue Router officiels : conventions communes
- Pages chargées à la demande : premier chargement léger

POURQUOI C'EST MOINS BIEN

- Moins demandé sur le marché que React (17,6 % contre 44,7 %)
- Écosystème de bibliothèques plus petit que celui de React

Vue vs React vs Angular

CRITÈRE	VUE	REACT	ANGULAR
Nature	Framework progressif	Bibliothèque UI	Framework complet
Mise à jour de l'écran	Automatique (ref)	Via les hooks (useState)	Via les signals
État & routing	Pinia + Router	À composer	Intégrés
Part du marché	17,6 % (SO 2025)	44,7 %	18,2 %
Courbe d'apprentissage	Faible	Moyenne	Élevé

Pourquoi Vue ? Il est léger (66,7 Ko compressés pour ma page d'entrée) et progressif : je n'ajoute que ce dont j'ai besoin. Angular vise des projets très complets, encadrés dès le départ, ce que ce projet ne demandait pas. React reste un choix valable, mais je l'ai déjà pratiqué.

Réactivité, composants, état

RÉACTIVITÉ

Quand une donnée change, l'écran se met à jour tout seul.

PINIA

Un endroit unique où sont rangées les données de l'appli, le "store".

veille-front/src/stores/board.ts (lignes 76-88)

```
76     card.listId = listId
77     target.splice(position, 0, card)
78     target.forEach((c, i) => (c.position = i))
79     if (sourceListId !== listId) {
80         cardsOf(sourceListId).forEach((c, i) => (c.position = i))
81     }
82
83     try {
84         await cardsApi.move(id, listId, position)
85     } catch (error) {
86         cards.value = snapshot
87         throw error
88     }
```

NestJS dans mon appli : ce que ça donne

DANS MON PROJET

3 modules : Auth, Lists, Cards

3 entités (tables) : User, List, Card

15 routes, documentées sur /api

2 guards : connexion (401), droits (403)

5 rôles et 7 permissions

Validation des données par des DTO

POURQUOI C'EST BIEN

- Architecture modulaire : module, controller, service
- Sécurité déclarative : guards et DTO, avant le code métier
- Swagger généré à partir des décorateurs
- TypeScript comme le front : un seul langage

POURQUOI C'EST MOINS BIEN

- Injection de dépendances et décorateurs à expliquer à l'équipe
- Moins répandu : NestJS 6,7 % contre Spring Boot 14,7 % (SO 2025)

NestJS vs Symfony vs Spring Boot

CRITÈRE	NESTJS · pratiqué	SYMFONY · lu	SPRING BOOT · lu
Langage	TypeScript	PHP	Java
Idée principale	Code rangé en modules	Briques réutilisables	Tout préconfiguré, démarrage rapide
Documentation de l'API	Lit les routes dans le code (@nestjs/swagger)	Via un module à ajouter (NelmioApiDocBundle)	Via une librairie à ajouter (springdoc-openapi)
Point fort	Même langage que le front	Très complet pour le web	Très robuste pour les grosses applis
Part du marché	6,7 % (SO 2025)	4 %	14,7 %

Pourquoi NestJS ? Il organise le code en modules (controller, service) et la sécurité en guards, et il est en TypeScript comme mon front : un seul langage pour toute l'équipe.

Qui a le droit de faire quoi

MODULES

Le code est rangé par thème : Auth, Lists, Cards.

INJECTION

Nest fournit lui-même les objets dont une classe a besoin.

GUARD ET DTO

Un guard vérifie qui a le droit. Un DTO vérifie que les données reçues sont valides.

LIMITE

Plus de fichiers et de notions à apprendre au début.

veille-back/src/auth/permissions.guard.ts (lignes 22-37)

```

22  canActivate(context: ExecutionContext): boolean {
23    const required = this.reflector.getAllAndOverride<Permission[] | undefined>(
24      PERMISSIONS_KEY,
25      [context.getHandler(), context.getClass()],
26    );
27    if (!required?.length) {
28      return true;
29    }
30    const user = context.switchToHttp().getRequest<{ user?: User }>().user;
31    const allowed =
32      !!user && required.every((p) => hasPermission(user.role, p));
33    if (!allowed) {
34      throw new ForbiddenException('Votre rôle ne permet pas cette action');
35    }
36    return true;
37  }

```

Swagger : la documentation de l'API

Auth		^
POST	/api/auth/register	v
POST	/api/auth/login	v
GET	/api/users/me	🔒 v
GET	/api/users	🔒 v
PATCH	/api/users/{id}	🔒 v
Users		^
GET	/api/users	🔒 v
PATCH	/api/users/{id}	🔒 v
Lists		^
GET	/api/lists	🔒 v
POST	/api/lists	🔒 v
PATCH	/api/lists/{id}	🔒 v
DELETE	/api/lists/{id}	🔒 v

À QUOI ÇA SERT

- Voir toutes les routes de l'API au même endroit
- Tester une route sans écrire de code, avec le token JWT
- Servir de contrat entre le front et le back

COMMENT C'EST FABRIQUÉ

- Nest lit les décorateurs du code : @Get, @Post, @ApiProperty...
- La documentation suit donc le code
- Le cadenas = route protégée par un JWT

DÉCISION

Vue + NestJS : cohérence avant préférence

Pourquoi je choisis cette stack

01 - Un seul langage

Du TypeScript sur le front et sur l'API : une seule équipe, un seul langage à apprendre.

02 - Code bien rangé

Vue et NestJS ont chacun une façon standard de ranger le code : un nouveau développeur sait où chercher.

03 - Front et API séparés

Le front ne parle qu'à l'API : on peut changer l'un sans toucher l'autre.

AFFINITÉ PERSONNELLE → Vue pour sa légèreté et sa prise en main progressive ; Nest pour sa structure.

Les versions et la maintenance

Vue 3

Version actuelle : Vue 3.5 (stable).
À venir : Vue 3.6, dans la même famille.

NestJS

Version actuelle : NestJS 12 (celle de mon projet).
Rythme : une version majeure tous les 1 à 2 ans.

Comment je maintiens

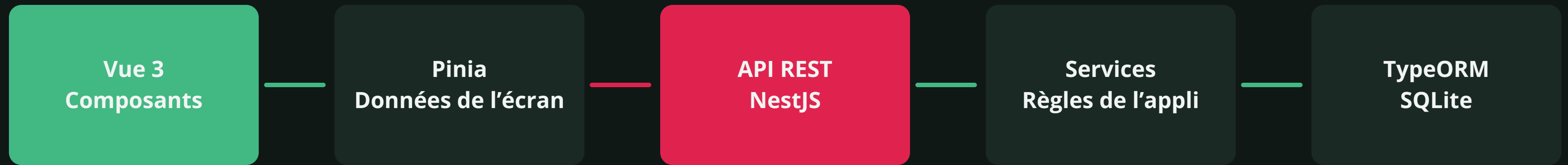
Le fichier package-lock.json fige les versions installées. Je mets à jour régulièrement et je lis les notes de version avant un changement majeur.

Déplacer une carte

DÉMO



Comment l'appli est construite



REQUÊTE →

← RÉPONSE / ÉTAT SYNCHRONISÉ

Le front (Vue) ne parle jamais à la base : il appelle l'API.

L'API (NestJS) vérifie les droits, valide les données, puis lit ou écrit dans la base.

Modèle : une SPA (une seule page dans le navigateur) + une API REST.

Changer de base de données : 1 fichier côté API (app.module.ts), aucun côté front.

APPRENTISSAGE

Ma courbe d'apprentissage

Ce qui est facile, ce qui demande du temps, et ce que ça change pour l'équipe.

**Vue
facile**

template proche du
HTML, SFC

**Vue
à creuser**

ref, computed et le store
Pinia

**NestJS
facile**

code rangé en modules,
Swagger

**NestJS
à creuser**

injection de
dépendances,
décorateurs

Équipe

un seul langage :
TypeScript

Mener le projet en temps limité

MUST - 31 pts (69 %)

Init (3) · Swagger (2) · Inscription (3) · Connexion (2) · Modifier un utilisateur (3) · Lister listes (1) · Créer liste (2) · Supprimer liste (2) · Lister cartes (1) · Créer carte (2) · Modifier carte (3) · Supprimer carte (1) · Tests (2) · Rapport (3) · README (1)

SHOULD - 4 pts (9 %)

Profil (1) · Modifier une liste (2) · Récupérer une carte (1)

COULD - 10 pts (22 %)

Tests API avancés (3) · Étude de performance (2)

WON'T - CETTE FOIS

Hors périmètre : temps réel · refresh token · limitation des tentatives de connexion

Ma décision

JE RECOMMANDE

Vue 3 + NestJS

Pourquoi : un seul langage, du code bien rangé, et une équipe opérationnelle vite.

JE CHANGE D'AVIS SI...

- Recruter devient difficile (Vue est moins demandé que React)
- L'application prend beaucoup plus d'ampleur

Marché (Stack Overflow 2025) : React 44,7 % · Angular 18,2 % · Vue 17,6 %

SOURCES

D'où viennent mes informations

DOCUMENTATIONS OFFICIELLES

Vue 3 : vuejs.org

NestJS et Swagger : docs.nestjs.com

PrimeVue : primevue.org

Code du projet : github.com/MathiasLeVrai/veille-techno-backend

ENQUÊTES ET COMPARATIFS

Stack Overflow Developer Survey 2025 : usage des technologies

<https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language-language-prof>

<https://2025.stateofjs.com/en-US/libraries/>

dyma.fr : « Quel framework choisir : Angular, Vue.js ou React ? »

<https://dyma.fr/blog/quel-framework-choisir-angular-vue-js-ou-react/>